

Amdox Task Manager

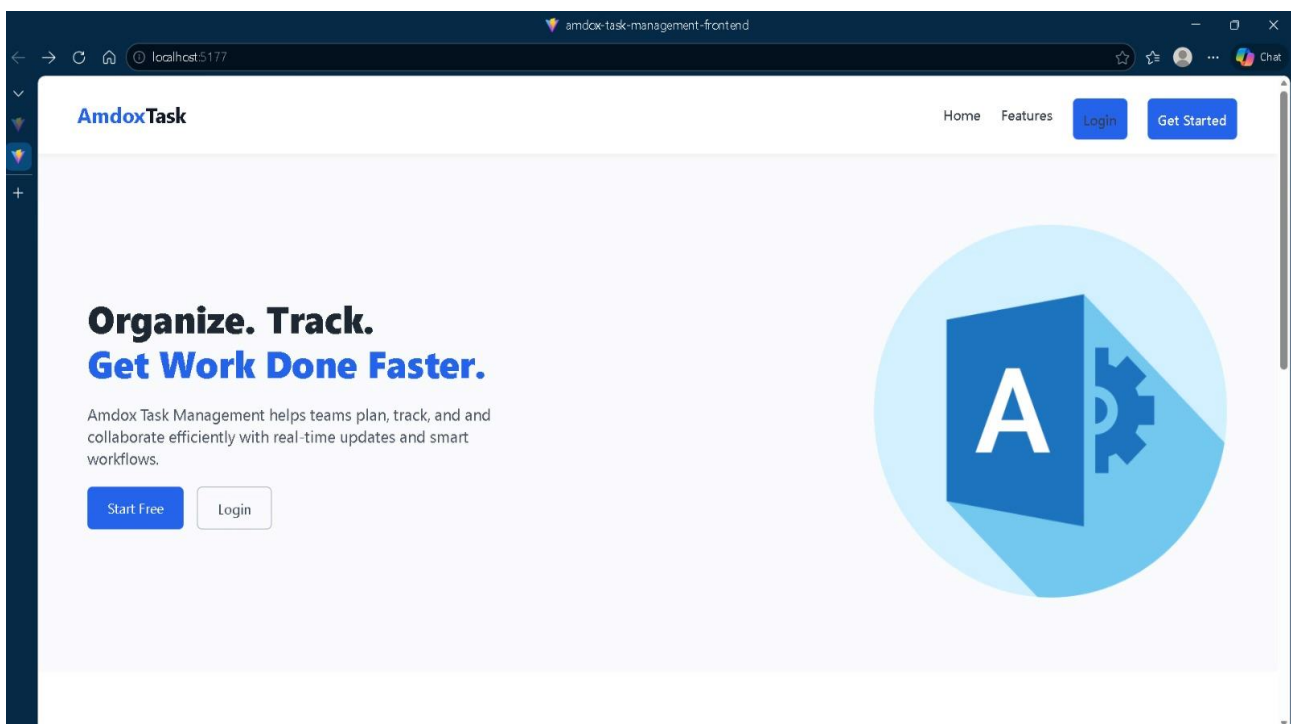
A Web-Based Task Management System

Index

1	Introduction
2	Project Objectives
3	System Architecture
4	Technology Stack
5	Functional Modules
6	Database Design
7	Frontend Implementation
8	Backend Implementation
9	API Testing with Postman
10	Screenshots Section
11	Future Enhancements
12	Conclusion

1. Introduction

The Amdox Task Manager is a web-based application designed to help teams organize and manage project tasks efficiently. The system provides functionality for task creation, workflow tracking, and board-based task organization similar to modern project management platforms. The application uses React for the frontend, Spring Boot for backend services, and MySQL for data storage.



AmdoxTask Home Page

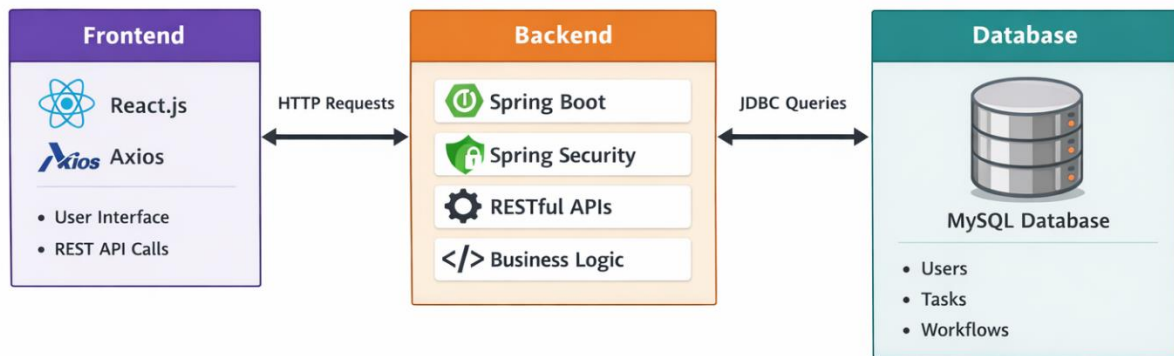
2. Project Objectives

- ✓ Build a task management platform for tracking work items.
- ✓ Implement secure authentication and user management.
- ✓ Allow users to create, update, and track tasks.
- ✓ Provide workflow-based task tracking.
- ✓ Develop a responsive and interactive user interface.

3. System Architecture

The system follows a three-tier architecture consisting of a presentation layer, application layer, and data layer. This structure improves maintainability, scalability, and separation of concerns.

Amdox Task Manager Architecture



4. Technology Stack

Frontend:

- React.js – For building a dynamic, component-based user interface
- Axios – For making HTTP requests to the backend REST APIs
- Bootstrap / Tailwind CSS – For responsive and modern UI design
- React Router – For client-side routing and navigation

Backend:

- Spring Boot – For building RESTful APIs and application logic
- Spring Security – For authentication and role-based access control
- Maven – For dependency management and building the backend
- Java 17 – Core programming language for backend logic

Database:

- MySQL – Relational database to store users, tasks, and workflows
- JPA / Hibernate – ORM for database interaction

Testing & Tools:

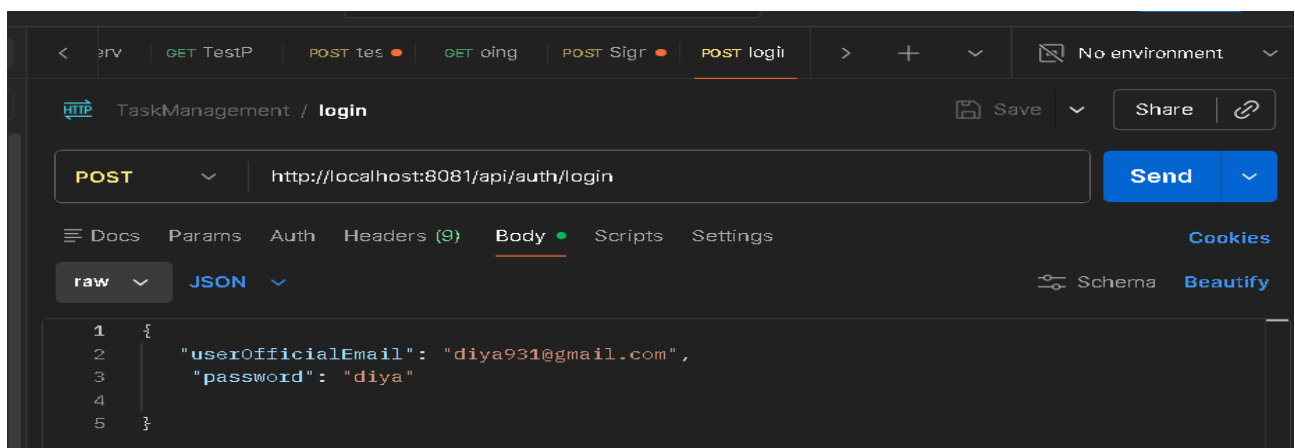
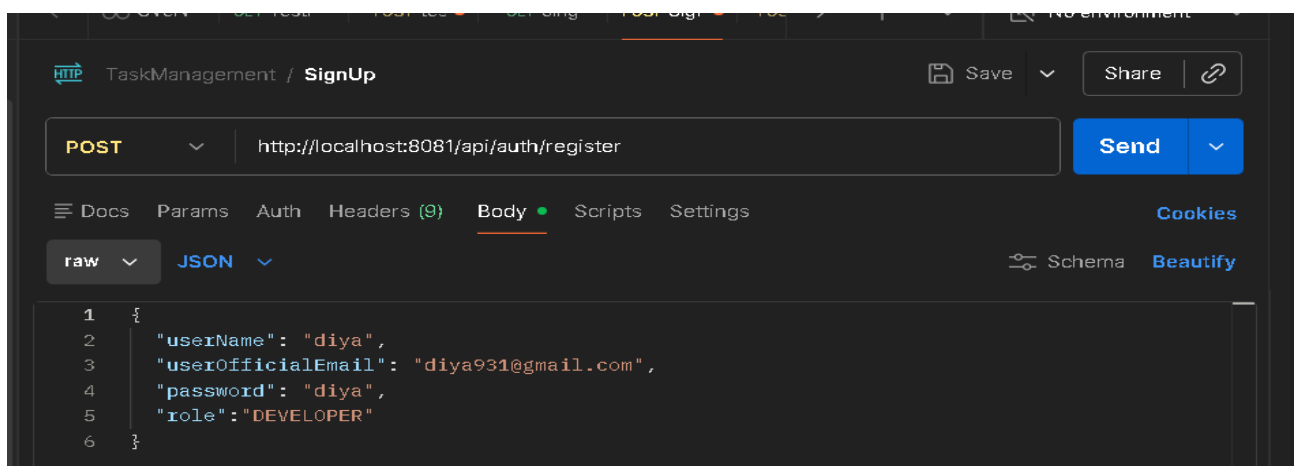
- Postman – API testing and development
- Git & GitHub – Version control and repository hosting
- IDE: IntelliJ IDEA / Eclipse

Layer	Technology
Frontend	React
Backend	Spring Boot
Database	MySQL
API Testing	Postman

5. Functional Modules

1. User Authentication Module :

The User Authentication Module is a critical part of the system that ensures secure access to the Amdox Task Manager. It handles user registration, login, and role-based access control, allowing different levels of permissions for admins and standard users. This module encrypts passwords using Spring Security and manages secure sessions to prevent unauthorized access. Beyond security, it also provides a seamless user experience for login and account management, forming the foundation for all other modules to function safely.



Welcome Back

Login to manage your tasks efficiently

Email

Password

Login

[Forgot Password?](#)

Don't have an account? [Sign Up](#)

Create Account

Join Amdox Task Management

User Name

Official Email

Password

Role

Select role

Register

Already have an account? [Login](#)

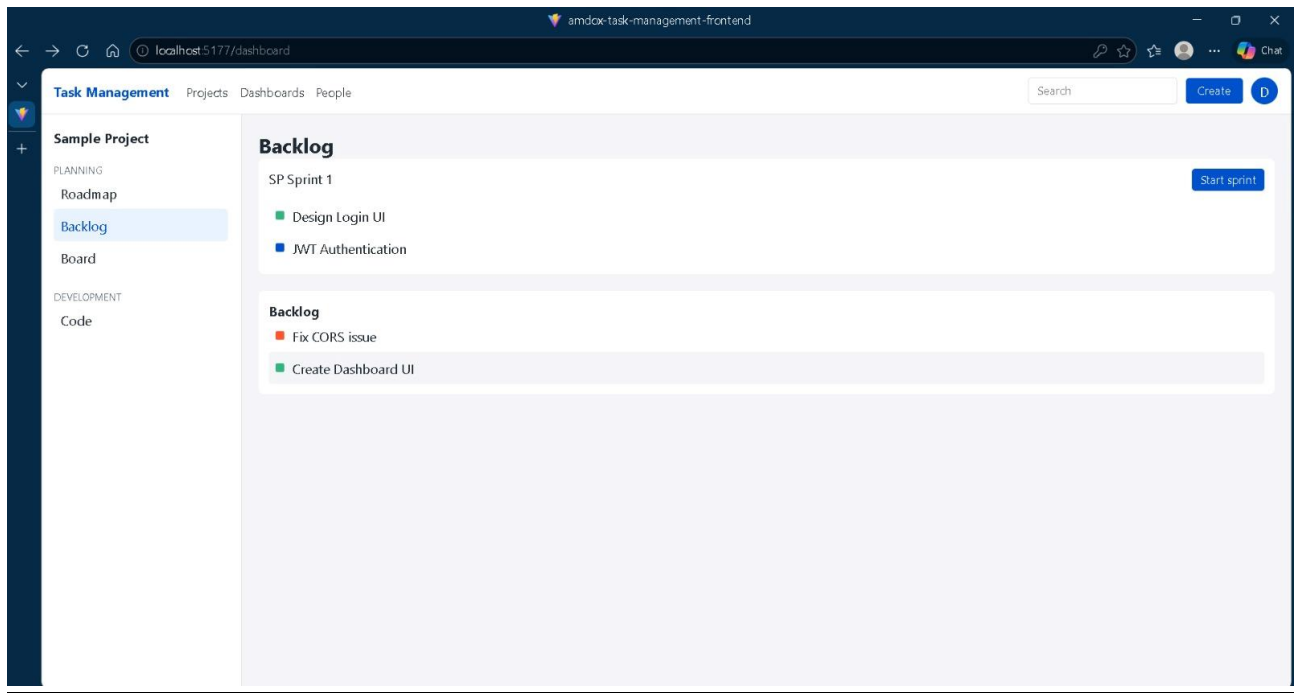
Forgot Password?

Enter your official email to reset your password

Official Email

Send Reset Link

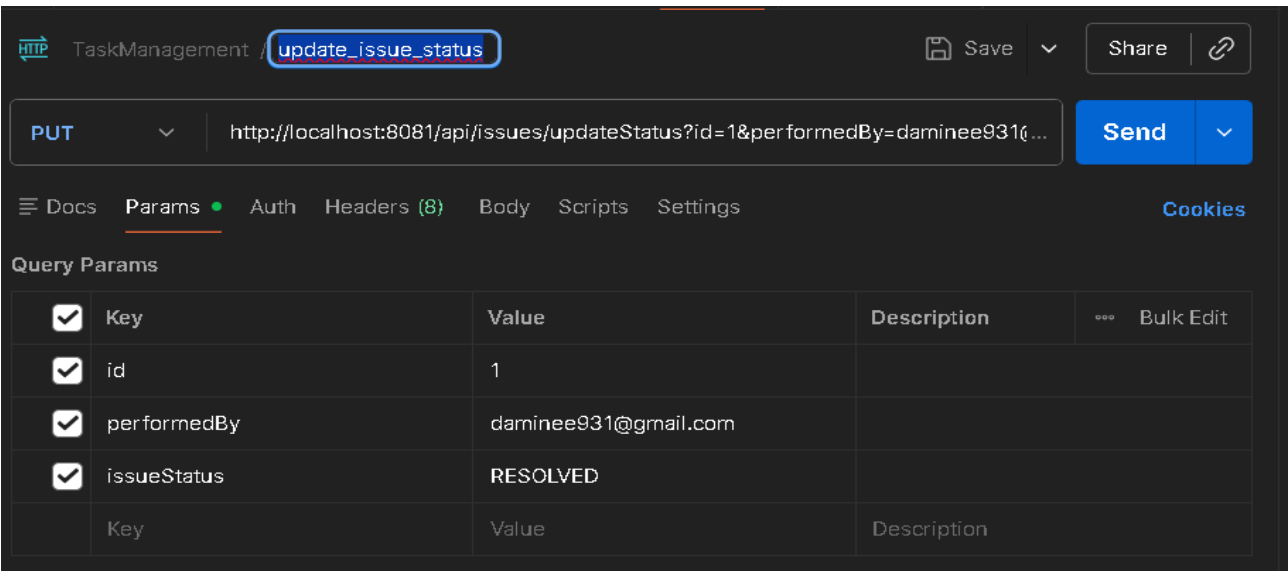
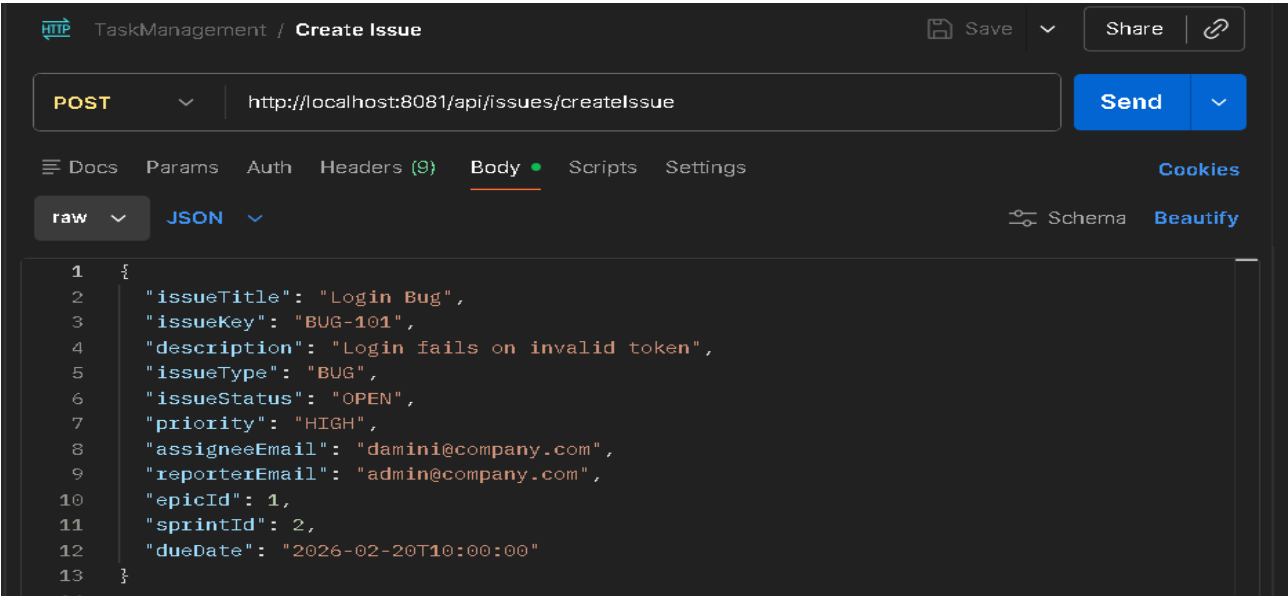
[Back to Login](#)

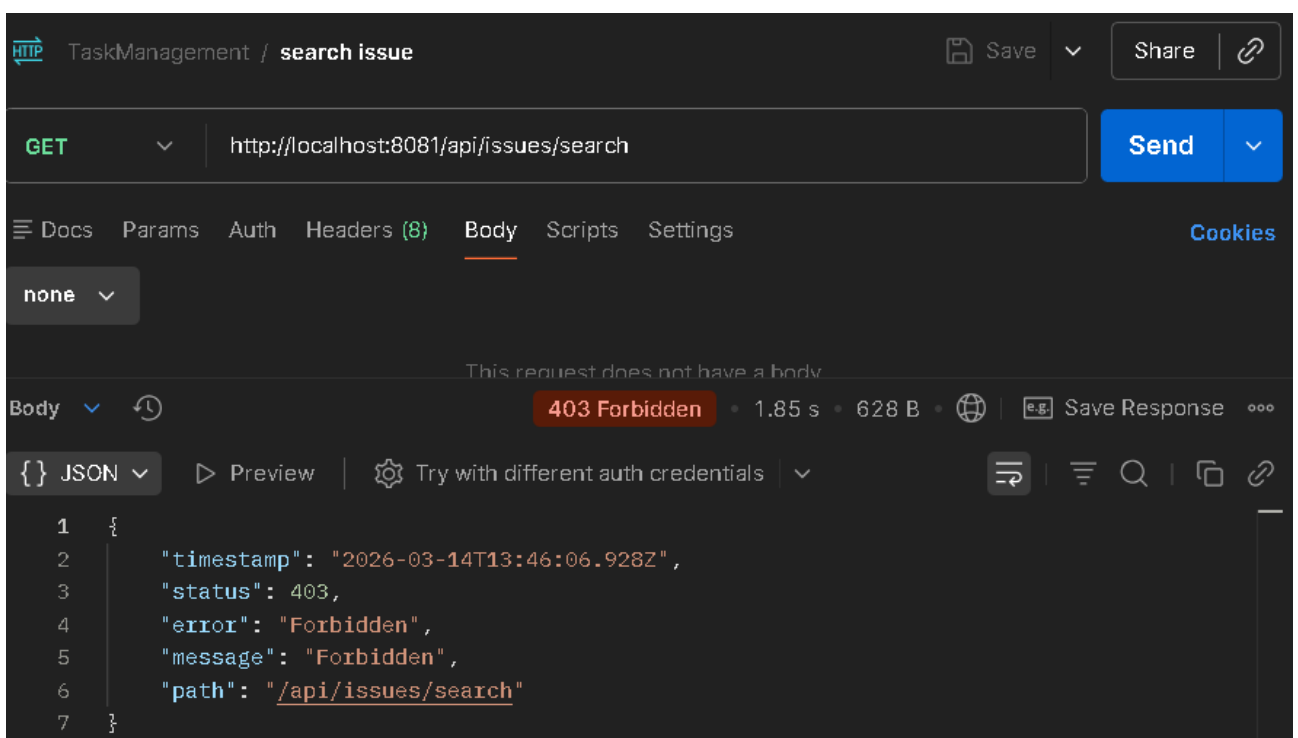
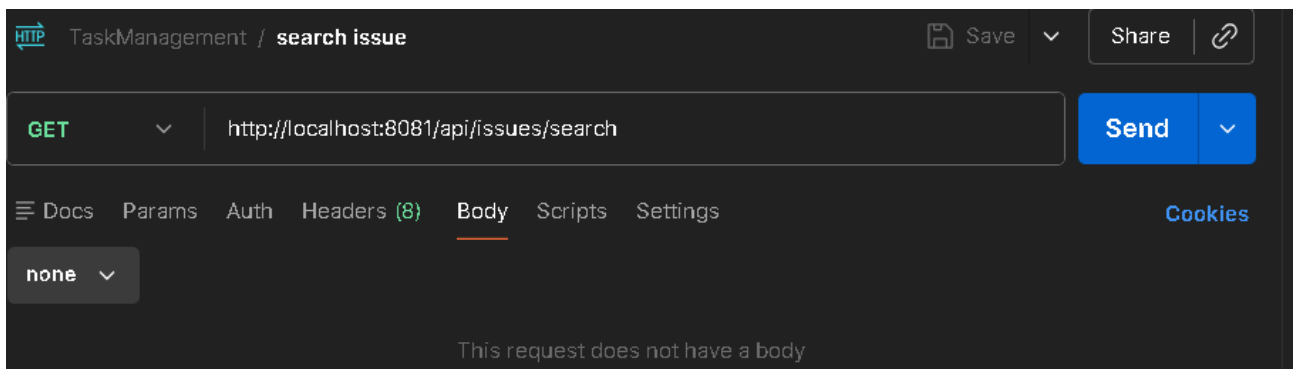
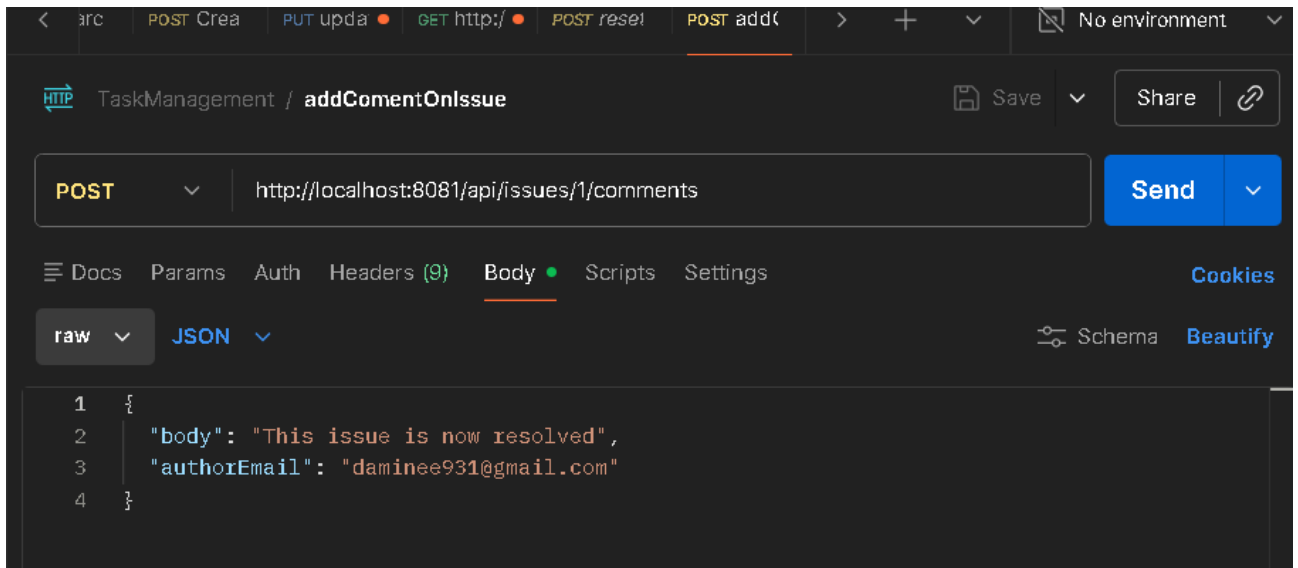


The Dashboard UI After User Login

IssueController Module

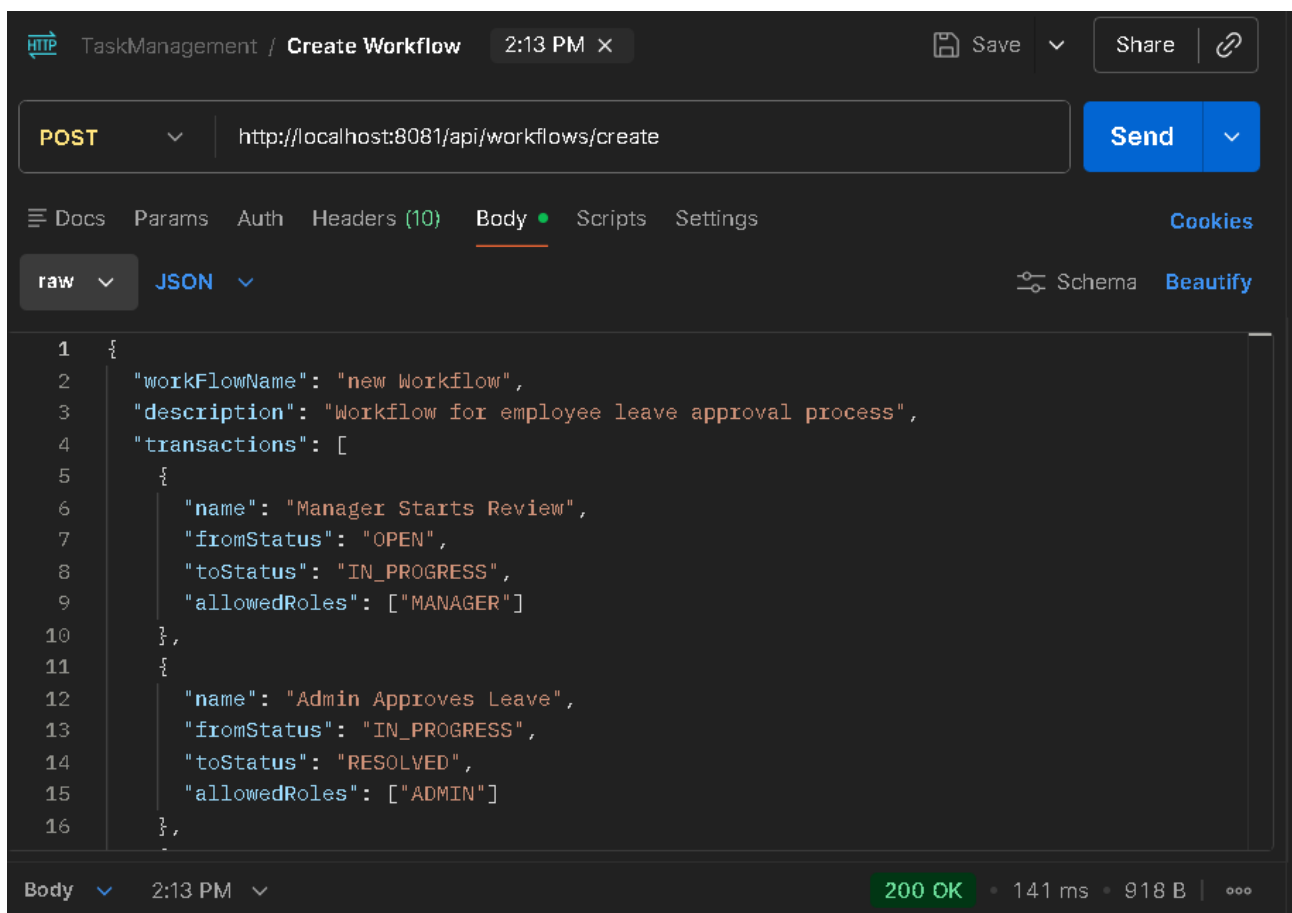
The IssueController Module serves as the backbone for task interaction in the system. It acts as the central point for handling all HTTP requests related to tasks or issues, ensuring that data sent from the frontend is correctly validated and processed by the backend services. By coordinating with the Task Management and Workflow Management modules, it guarantees that tasks follow the correct lifecycle and status transitions. This module essentially bridges the frontend and backend, making sure that all operations are consistent, reliable, and maintainable.





Workflow Management Module

The Workflow Management Module defines and controls the movement of tasks through various stages such as “Open,” “In Progress,” “Review,” and “Done.” It allows configurable workflows for different projects and enforces rules for task transitions, ensuring consistency and preventing errors in task progression. By providing clear visibility into task states, this module helps teams prioritize work, maintain order, and monitor project progress effectively. It also integrates with Task Management and Board Management modules to provide a seamless user experience.



TaskManagement / **Get All Workflows** 10:37 AM

GET

http://localhost:8081/api/workflows/all

Send

Docs

Params

Auth

Headers (10)

Body

Scripts

Settings

Cookies

Query Params

	Key	Value	Description	...	Bulk Edit
--	-----	-------	-------------	-----	-----------

Body 10:37 AM 200 OK • 25 ms • 744 B

TaskManagement / **Update Workflow by id**

PUT

http://localhost:8081/api/workflows/update/3

Send

Docs

Params

Auth

Headers (10)

Body

Scripts

Settings

Cookies

raw

JSON

Schema

Beautify

```
1 {
2   "workFlowName": "Updated Leave Approval",
3   "description": "Updated workflow for leave approval process",
4   "transactions": [
5     {
6       "fromStatus": "OPEN",
7       "toStatus": "IN_PROGRESS",
8       "allowedRoles": ["MANAGER"]
9     },
10    {
11      "fromStatus": "IN_PROGRESS",
12      "toStatus": "RESOLVED",
13      "allowedRoles": ["ADMIN"]
14    }
15  ]
16 }
```



TaskManagement / Get workflow by id



Save



Share



GET



http://localhost:8081/api/workflows/3

Send



Docs

Params

Auth

Headers (9)

Body

Scripts

Settings

Cookies

Body



200 OK

• 664 ms • 775 B •



Save Response



{}

JSON



Preview

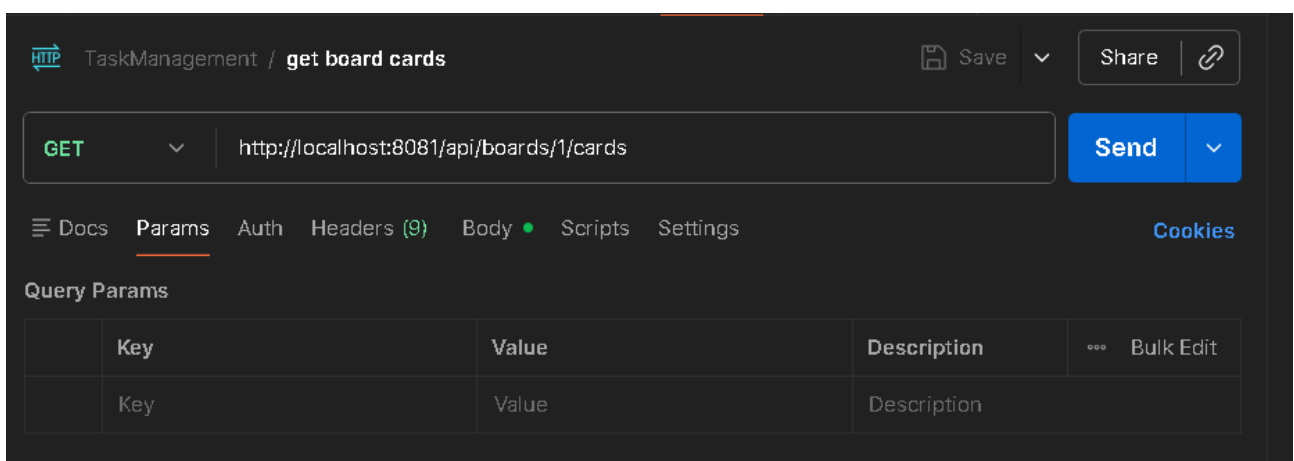
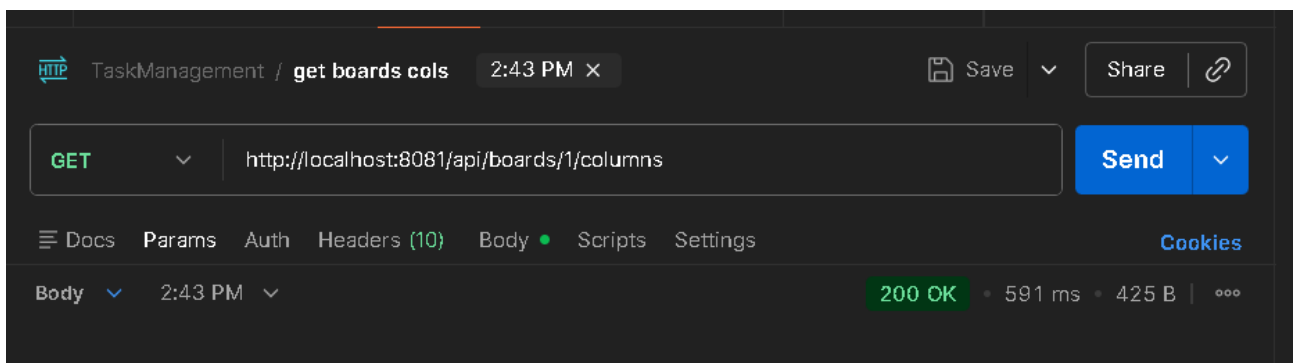
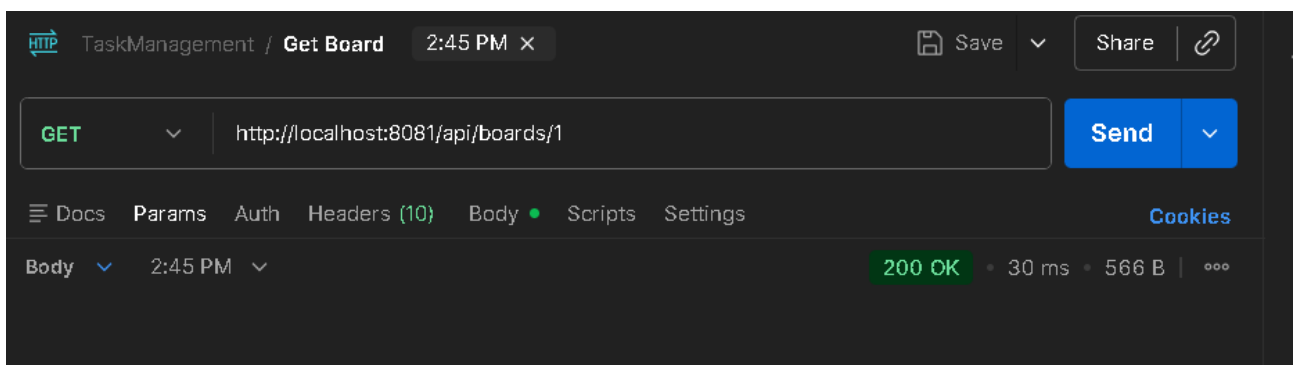
Visualize



```
1  {
2    "id": 3,
3    "workFlowName": "Updated Leave Approval",
4    "description": "Updated workflow for leave approval process",
5    "createdAt": "2026-03-14T13:59:08.2533",
6    "transactions": [
7      {
8        "id": 5,
9        "fromStatus": "IN_PROGRESS",
10       "toStatus": "RESOLVED",
11       "name": null,
12       "allowedRoles": [
13         "ADMIN"
14       ]
15     },
16     {
17       "id": 4,
18       "fromStatus": "OPEN",
19       "toStatus": "IN_PROGRESS",
20       "name": null,
21       "allowedRoles": [
22         "MANAGER"
23       ]
24     }
25   ]
26 }
```

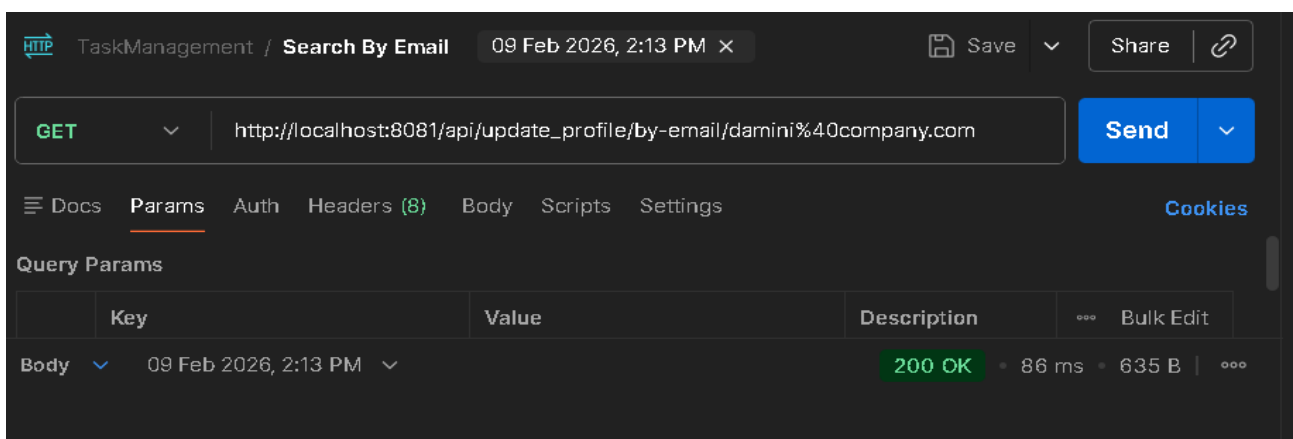
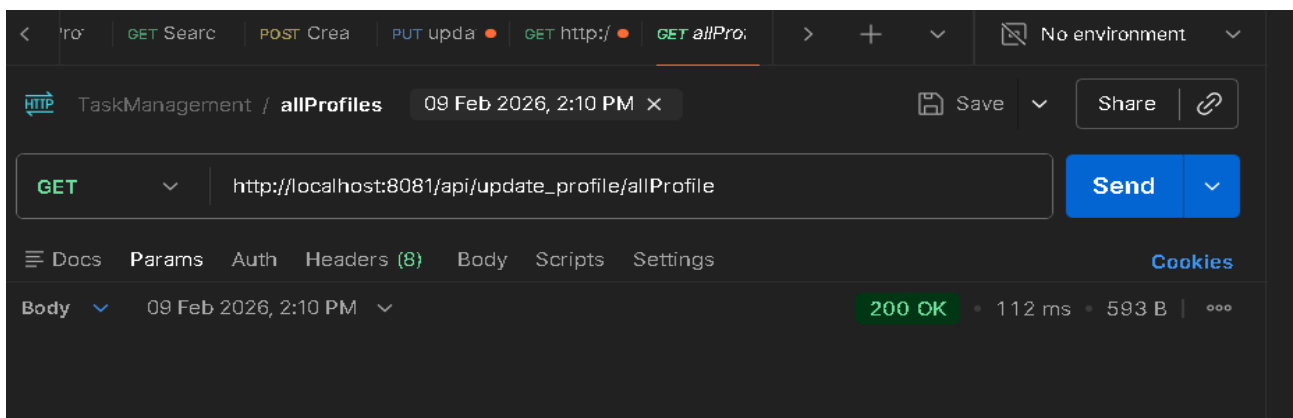
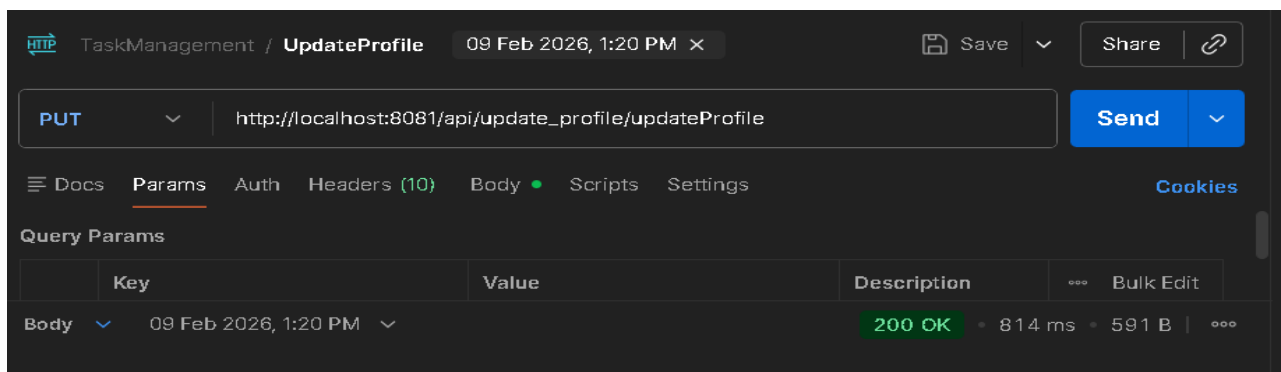
Board Management Module

The Board Management Module offers a visual and interactive way to organize and track tasks. Users can create boards for projects or teams, define columns for task statuses, and arrange tasks within those columns. It supports drag-and-drop functionality for easy task movement, position-based ordering, and a clear overview of ongoing work. This module enhances productivity by giving users a bird's-eye view of tasks and workflows, helping teams coordinate and collaborate efficiently.



User Profile Management Module

The User Profile Management Module manages all personal information and settings of users. It allows users to update their profile details, change passwords, and customize account preferences. Beyond account management, it provides dashboards and summaries of tasks assigned to the user, task completion statistics, and activity logs. This module ensures that each user has a personalized experience while maintaining accountability and transparency within the system.

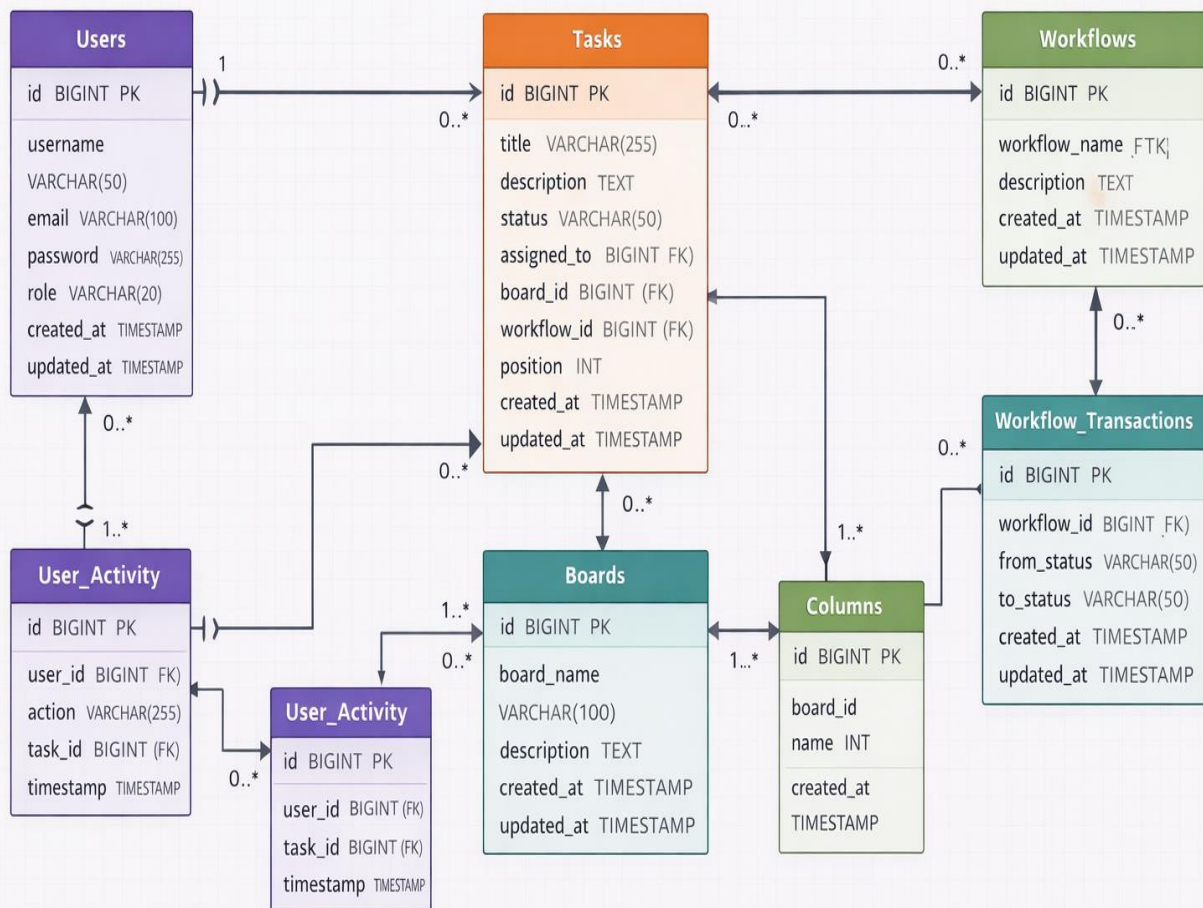


6. Database Design

The system uses MySQL to store user data, boards, workflows, and tasks. The database schema ensures data consistency and efficient retrieval.

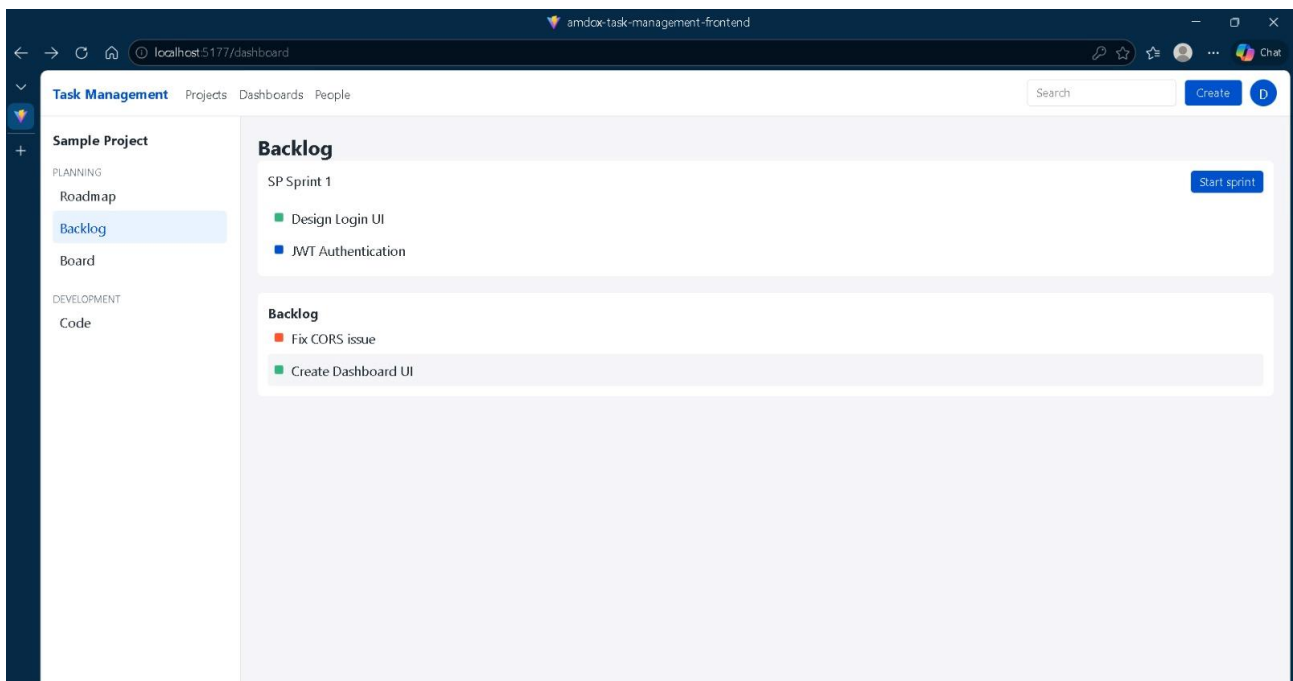
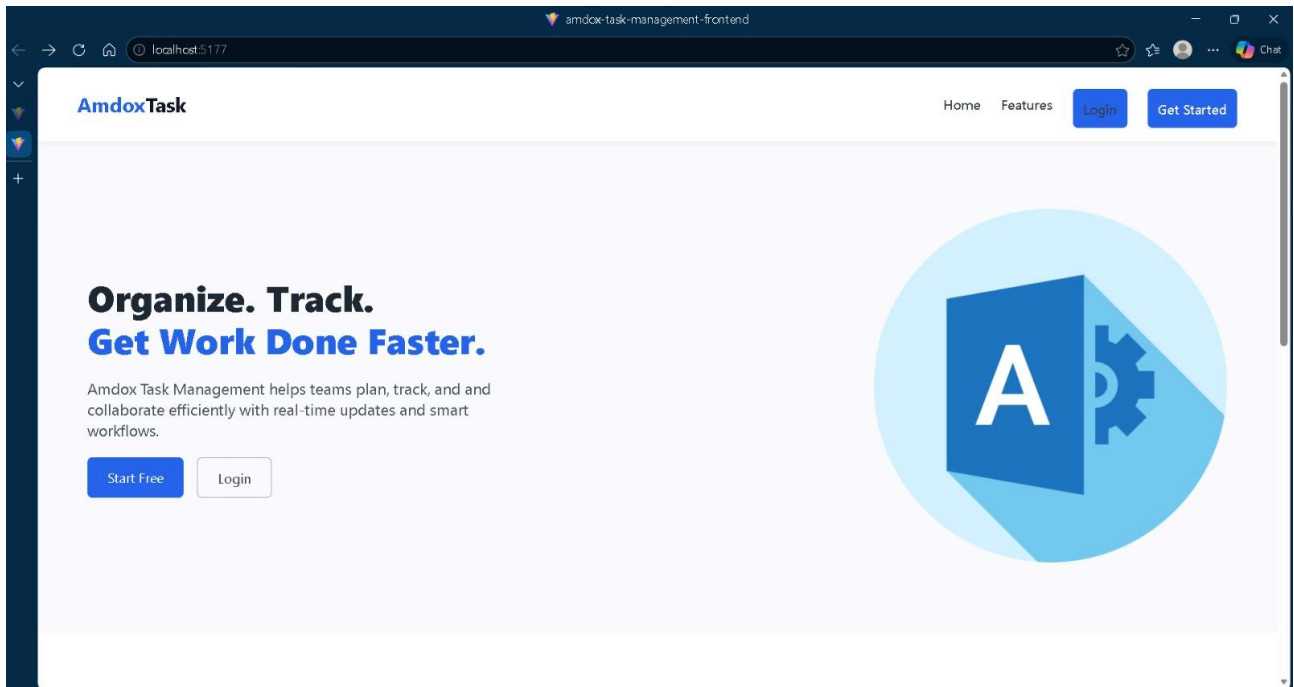
Sc

ER Diagram for Amdox Task Manager



7. Frontend Implementation

The frontend interface is developed using React. It provides interactive components for managing tasks, boards, and workflows. Axios is used to communicate with backend APIs.

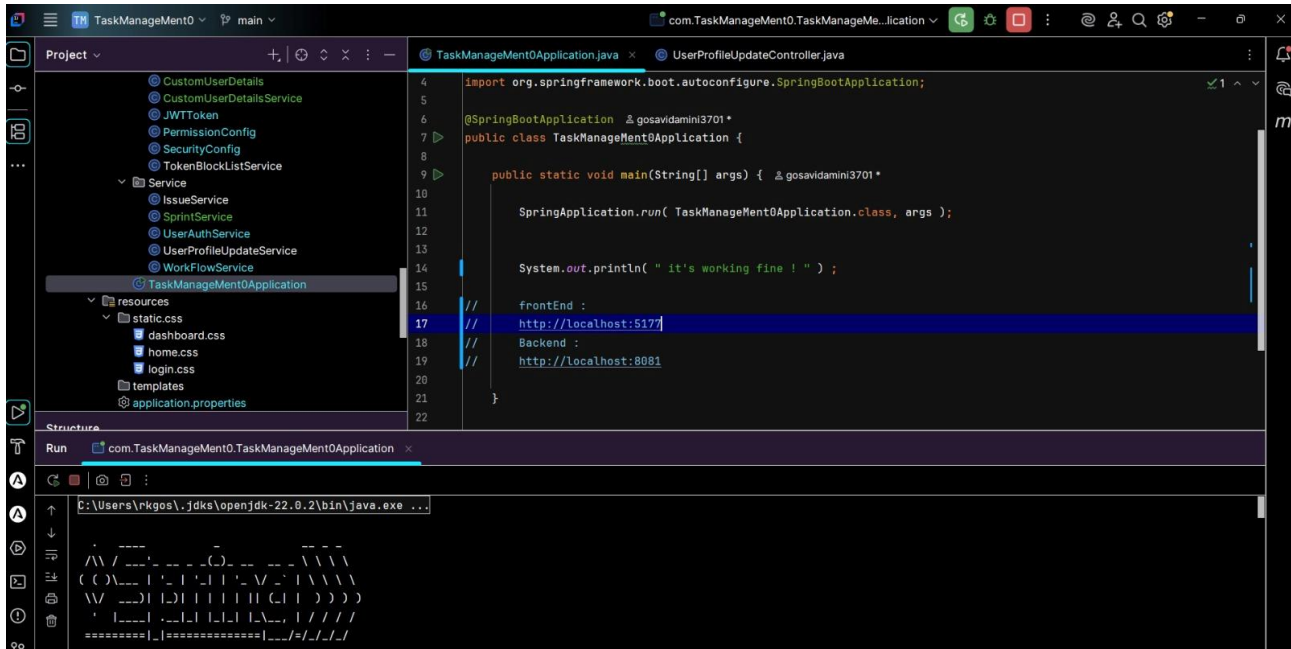


```
File Edit Selection View ... FrontEnd
EXPLORER
FRONTEND
  amdox-task-management-frontend
    node_modules
    public
    src
      api
        authApi.js
        axiosInstance.js
        taskApi.js
      components
        auth
          ForgotPassword.css
          ForgotPassword.jsx
          Login.css
          Login.jsx
          Register.css
          Register.jsx
          ResetPassword.css
          ResetPassword.jsx
        common
          Button.jsx
    OUTLINE

authApi.js
amdox-task-management-frontend > src > api > authApi.js > ...
1 import axios from "axios";
2
3 const BASE_URL = "http://localhost:8081/api/auth";
4
5 // If you want session/cookie-based auth
6 axios.defaults.withCredentials = true;
7
8 // Register
9
10
11 export const registerUser = async ({ userName, userOfficialEmail, password, role })
12 => {
13   return axios.post(
14     `${BASE_URL}/register`,
15     { userName, userOfficialEmail, password, role },
16     {
17       headers: {
18         "Content-Type": "application/json",
19       },
20       withCredentials: true
21     }
22   );
23 }
```

8. Backend Implementation

The backend services are implemented using Spring Boot. RESTful APIs handle authentication, task management, workflow processing, and board operations.



Spring Boot code

```
// CORS configuration for React + Angular frontends
@Bean no usages @gosavidamini3701
public CorsConfigurationSource corsConfigurationSource() {
    CorsConfiguration configuration = new CorsConfiguration();

    // Allowed origins (your frontend dev servers)
    String number = "5177";
    configuration.setAllowedOrigins(List.of(
        "http://localhost:" + number, // React dev URL
        "http://localhost:4200" // Angular dev URL
    ));

    // Allowed HTTP methods
    configuration.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE", "OPTIONS"));

    // Allowed headers
    configuration.setAllowedHeaders(List.of("*"));

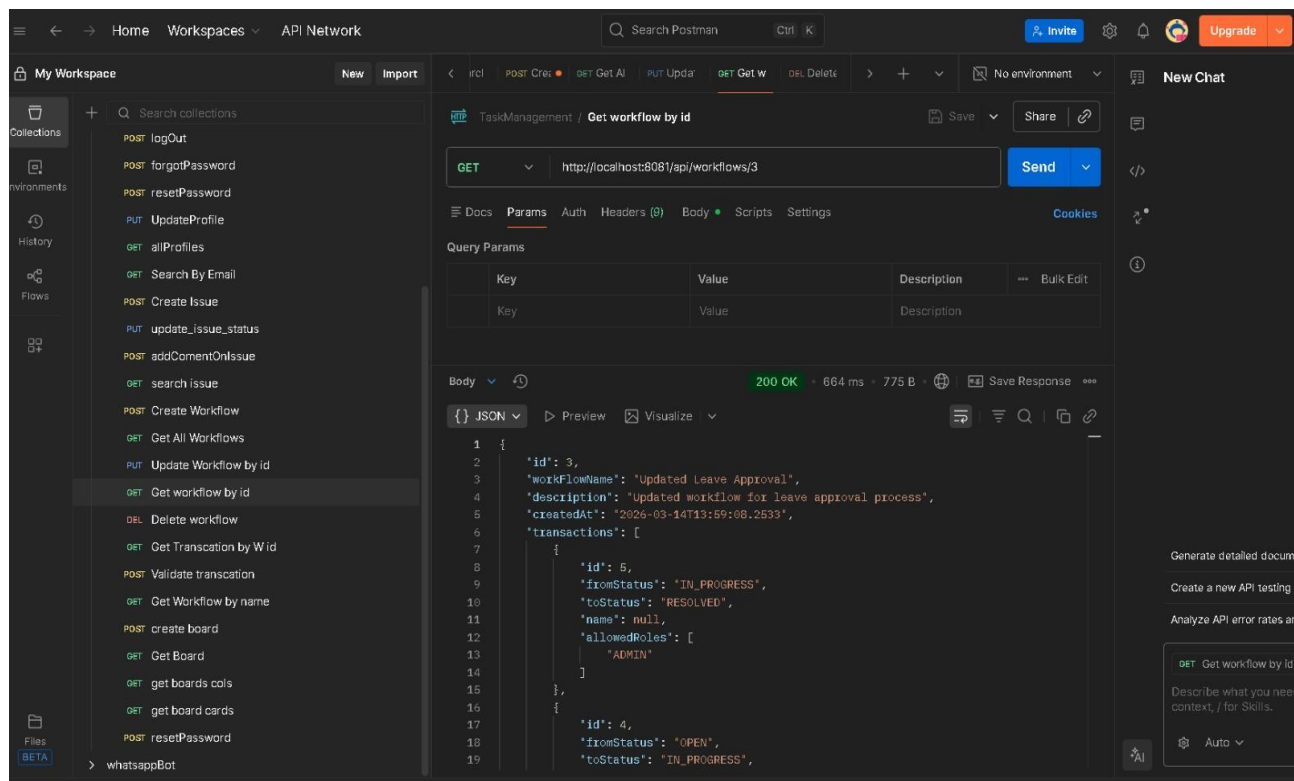
    // Allow credentials (cookies/sessions)
    configuration.setAllowCredentials(true);

    UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
    source.registerCorsConfiguration(pattern: "**", configuration);

    System.out.println(" CORS configured for frontend origins.");
}
```

9. API Testing with Postman

All REST APIs were tested using Postman to validate request handling, response structures, and authentication mechanisms.



Screenshot Placeholder – Postman API Collection

✓ **Future Enhancements**

- Role-based access control
- Drag-and-drop task management
- Real-time notifications
- Team collaboration features

11. Conclusion

The Amdox Task Manager demonstrates a full-stack web application integrating React, Spring Boot, and MySQL to manage project tasks efficiently. The system provides a scalable structure that can be extended with additional enterprise features.